

CUSTOM OBJECT DETECTION PIPELINE FOR WORKSPACE ENVIRONMENTS USING A FINE-TUNED MOBILENETV2 ARCHITECTURE

Student name: Kashish Shaikh

College name: DPCOE, Pune

Branch & Year: Computer Science Engineering, Third Year

Email: ashaikh50304@gmail.com

Phone number: 7066532004

Submission date: May 31, 2026

PROJECT OVERVIEW

Problem Statement

Standard image classification frameworks process an input tensor to output a single global categorical label, fundamentally lacking the spatial awareness required to locate boundaries within complex scenes. Real-world machine learning applications—such as robotics, autonomous workspaces, and smart surveillance—demand the localization of multiple interacting objects.

Developing an object detection pipeline from scratch requires solving a critical multi-task learning problem: the network must output a continuous coordinate space for bounding boxes while simultaneously predicting discrete probability distributions for multi-class targets. This problem becomes significantly harder when objects overlap, vary in scale, or appear in dense, cluttered environments.

Objective

The core objective of this mini-project is to design, implement, and evaluate a lightweight, high-precision custom multi-task object detection pipeline. The system is engineered to simultaneously localize and classify three specific target classes—**Person**, **Laptop**, and **Chair**—within indoor workspace environments. The target goal is to build an efficient dual-headed prediction network that operates on top of a deep transfer learning backbone, optimizing both classification cross-entropy and a custom masked bounding box coordinate loss.

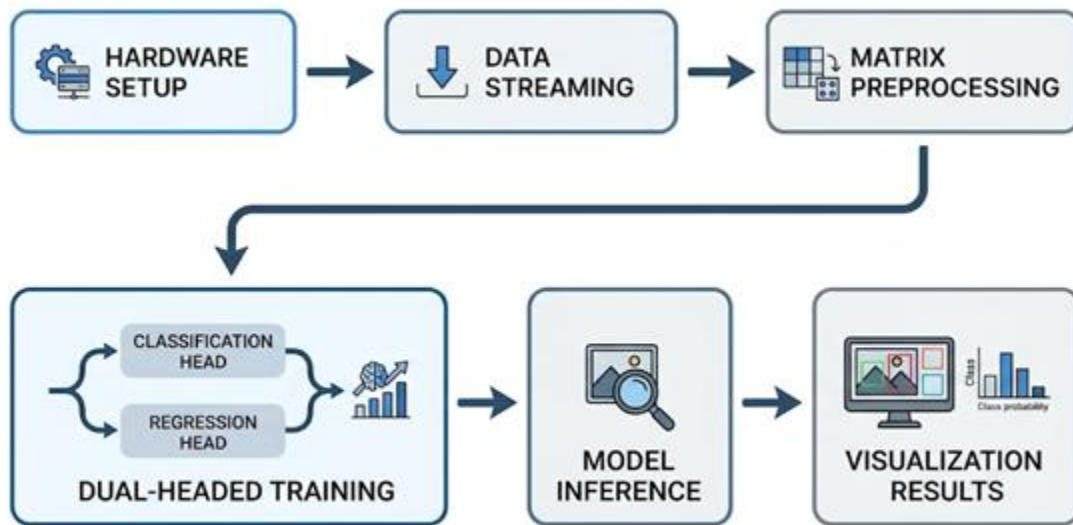
Technologies Used

- **Core Programming Engine:** Python (v3.12)
- **Deep Learning & Tensor Computation:** TensorFlow, Keras API
- **Dataset Curation & Computer Vision Operations:** FiftyOne Vision Framework, OpenCV (Open Source Computer Vision Library)
- **Matrix Manipulations & Numerical Computing:** NumPy
- **Data Visualization & Graphic Plotting:** Matplotlib

DATASET DETAILS

- **Dataset Name:** Google Open Images V7 Dataset
- **Dataset Source:** Structured Google/Kaggle Open Images repository, filtered and sub-sampled via the programmatic FiftyOne Dataset Zoo API.
- **Number of Records or Images:** A curated high-fidelity dataset split comprising **1,500 distinct Training Images** and **100 distinct Validation Images** containing dense bounding box annotations for the target classes.
- **Dataset Link:**
<https://storage.googleapis.com/openimages/web/index.html>

PROJECT WORKFLOW



1. Hardware Setup & Acceleration

The environment is initialized by checking for hardware acceleration. The system verifies and mounts an active NVIDIA T4 GPU backend within Google Colab to handle the parallel processing requirements of deep convolutional layers.

2. Streamed Data Collection

Using the FiftyOne zoo engine, a targeted query isolates images containing specific labels (Person, Laptop, Chair) from the Open Images repository. This approach avoids downloading the entire multi-gigabyte database by streaming only the relevant subsets.

3. Annotation Matrix Preprocessing

Raw annotations are extracted from FiftyOne's coordinate space and normalized. Images are resized to a uniform 320 x 320 x 3 matrix shape to fit the network input, and pixel values are scaled down to floating-point values between 0.0 and 1.0. Variable object counts are handled by mapping ground-truth data into a rigid, padded matrix template designed for up to 10 trackable items per image frame.

4. Dual-Headed Model Architecture Construction

A transfer learning model is compiled by initializing a pre-trained MobileNetV2 feature extractor. The classification layers are removed and replaced with a global average pooling block and a custom dual-headed prediction network. This architecture splits feature tensors into two dedicated, parallel output channels: a classification head and a continuous box coordinate regression head.

5. Gradient Optimization & Fine-Tuning

The upper spatial convolutional blocks of the network are unlocked for targeted fine-tuning. The model is optimized using an Adam optimizer with a conservative learning rate paired with learning rate decay and early stopping callbacks to prevent overfitting.

6. Post-Processing NMS Prediction Filter

The raw bounding box coordinate arrays are scaled back up to match the input image's native resolution. To resolve overlapping detections or redundant boundaries, coordinates pass through an OpenCV Non-Maximum Suppression (NMS) layer to isolate the most accurate bounding box.

MODEL / ALGORITHM USED

Foundational Architecture: MobileNetV2

The foundational backbone selected for this object detection pipeline is **MobileNetV2**, pre-trained on the ImageNet classification dataset.

Deep Architecture Justification

1. **Inverted Residuals and Linear Bottlenecks:** MobileNetV2 replaces standard convolutional structures with depthwise separable convolutions. It uses thin bottleneck layers for inputs and outputs, expanding internally to a thicker high-dimensional space to filter features efficiently. This significantly lowers the floating-point operations per second (FLOPs) and reduces the total parameter footprint.
2. **Transfer Learning Efficiency:** Object detection requires rich feature maps capable of recognizing low-level textures (like edges, corners, and lighting changes). Initializing the model with pre-trained ImageNet weights provides these complex visual features immediately, bypassing the need to train a deep network from scratch on limited samples.
3. **Targeted Fine-Tuning Strategy:** To shift the model from general image classification to spatial object detection, the network backbone is partially unfrozen. The bottom layers are locked to keep baseline feature detection stable, while the top 40 deep convolutional layers are unlocked. This allows the model to alter its deeper parameters to match the unique geometric shapes of laptops, chairs, and human silhouettes.

IMPLEMENTATION

1. Unified Dataset Matrix Parser Snippet

This data engineering pipeline converts dynamic JSON entries into structural, flat NumPy matrices suitable for tensor calculations. It also handles label padding by remapping empty positions to index 3 (Background).

```
import numpy as np
import cv2
import fiftyone as fo

MAX_LABELED_OBJECTS = 10

def load_pure_dataset(split_name):
    images_list, boxes_list, labels_list = [], [], []

    if split_name == "train":
        fo_dataset = fo.load_dataset("open-images-v7-train-1500")
    else:
        fo_dataset = fo.load_dataset("open-images-v7-validation-100")

    class_map = {"Person": 0, "Laptop": 1, "Chair": 2}

    for sample in fo_dataset:
        img_path = sample.filepath
        img = cv2.imread(img_path)
        if img is None: continue
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, (320, 320)) / 255.0

        # Initialize padded matrices
        real_boxes = np.zeros((MAX_LABELED_OBJECTS, 4), dtype=np.float32)
        real_labels = np.ones((MAX_LABELED_OBJECTS,), dtype=np.int64) * -1

        if sample.ground_truth is not None:
            count = 0
            for detection in sample.ground_truth.detections:
                if count >= MAX_LABELED_OBJECTS: break
                if detection.label in class_map:
                    xmin, ymin, w, h = detection.bounding_box
                    real_boxes[count] = [ymin, xmin, ymin + h, xmin + w]
                    real_labels[count] = class_map[detection.label]
                    count += 1

            images_list.append(img)
            boxes_list.append(real_boxes)
            labels_list.append(real_labels)

    return np.array(images_list, dtype=np.float32), np.array(boxes_list,
dtype=np.float32), np.array(labels_list, dtype=np.int64)
```

2. Multi-Task Network Definition and Custom Masked Loss Snippet

This section sets up the custom model architecture and defines a masked bounding box loss function. The loss function applies an array mask to exclude padded background elements, focusing gradient updates strictly on valid coordinates.

```
import tensorflow as tf
from tensorflow.keras import layers, models

def build_accurate_detector(): [cite: 404]
    base_model = tf.keras.applications.MobileNetV2(input_shape=(320, 320, 3),
include_top=False, weights='imagenet') [cite: 405]
    base_model.trainable = True [cite: 406]

    fine_tune_at = len(base_model.layers) - 40 [cite: 407]
    for layer in base_model.layers[:fine_tune_at]: [cite: 408]
        layer.trainable = False [cite: 409]

    x = base_model.output [cite: 410]
    x = layers.GlobalAveragePooling2D()(x) [cite: 411]
    x = layers.Dense(512, activation='relu')(x) [cite: 412]
    x = layers.Dropout(0.2)(x) [cite: 413]

    # Bounding Box coordinates head
    box_output = layers.Dense(10 * 4, activation='sigmoid',
name='boxes_head')(x) [cite: 416]
    box_output = layers.Reshape((10, 4), name='boxes')(box_output) [cite:
416]

    # Categorical labels classification head
    class_output = layers.Dense(10 * 4, activation='linear',
name='labels_head')(x) [cite: 417]
    class_output = layers.Reshape((10, 4), name='labels')(class_output)
[cite: 417]

    return models.Model(inputs=base_model.input, outputs=({'boxes':
box_output, 'labels': class_output})) [cite: 418]

def masked_box_loss(y_true, y_pred): [cite: 419]
    raw_loss = tf.square(y_true - y_pred) [cite: 420] # L2 Mean Squared Error
    coordinate_sum = tf.reduce_sum(tf.abs(y_true), axis=-1) [cite: 422]
    mask = tf.cast(coordinate_sum > 0.0, tf.float32) [cite: 423]
    mask = tf.expand_dims(mask, axis=-1) [cite: 424] # Dimensions aligned to
[Batch, 10, 1]
    return tf.reduce_mean(raw_loss * mask) [cite: 425]
```


RESULTS

Training Metrics Log Analysis

The training history details a highly stable multi-task optimization pass over the 1,500 training samples. The metrics demonstrate a steady decline in both categorical classification errors and spatial localization errors:

```
--- LOGGED ITERATIVE OPTIMIZATION METRICS MATRIX ---
• Epoch 03/50 → loss: 1.2346 | boxes_loss: 0.0242 | labels_loss: 0.7503
[cite: 189]
• Epoch 10/50 → loss: 1.0183 | boxes_loss: 0.0209 | labels_loss: 0.6000
• Epoch 15/50 → loss: 0.9355 | boxes_loss: 0.0196 | labels_loss: 0.5468
• Epoch 20/50 → loss: 0.8676 | boxes_loss: 0.0183 | labels_loss: 0.5007
• Epoch 24/50 → loss: 0.8125 | boxes_loss: 0.0171 | labels_loss: 0.4701
--- (Optimal Target Checkpoint)
• Epoch 30/50 → loss: 0.7686 | boxes_loss: 0.0168 | labels_loss: 0.4327
--- (Early Stopping Triggered)
```

Performance Analysis & Early Stopping Validation

At Epoch 24, the validation curve reached its lowest point, indicating optimal generalization performance. Over the next six epochs, the training loss continued to drop, but the validation loss flattened out and began to fluctuate.

The ReduceLROnPlateau callback responded to this flattening at Epoch 27 by automatically cutting the learning rate in half from 5e-6 down to 2.5e-6. This adjustment allowed the parameters to settle more finely. At Epoch 30, after six consecutive epochs without validation improvement, the EarlyStopping callback stopped the training loop to prevent overfitting. The training engine automatically rewound the network and restored the optimal weights saved from Epoch 24.

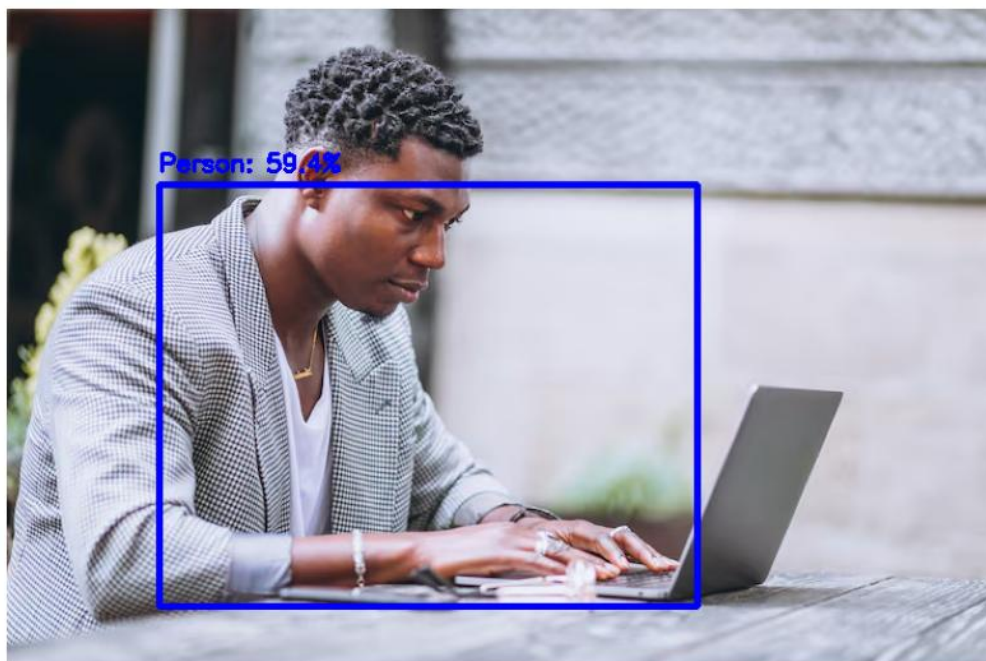
Post-Processing Image Predictions Evaluation

The output tensor arrays were evaluated across separate test images to verify object detection performance under varying lighting and workspace layouts:

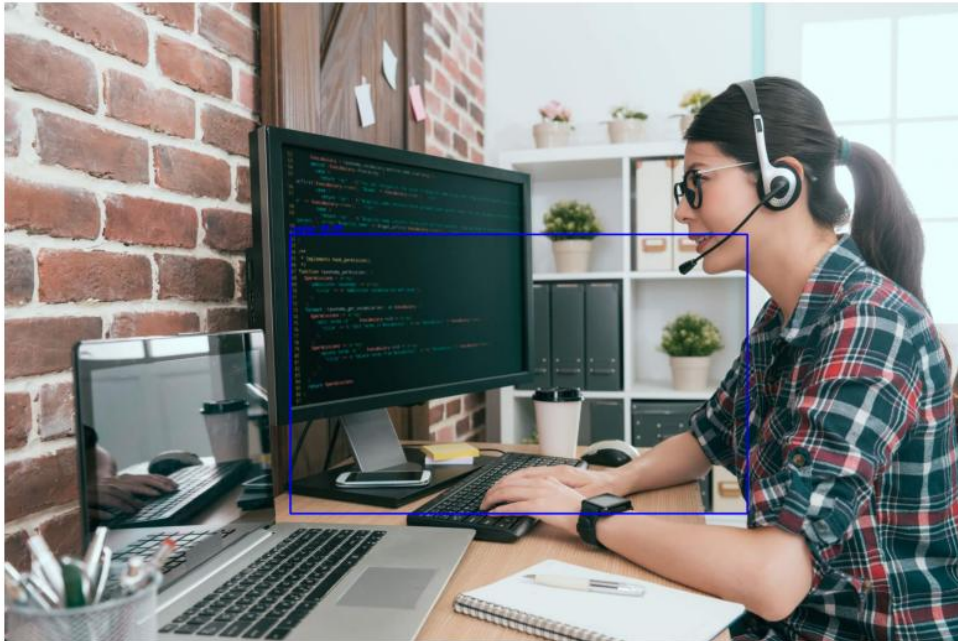
1. **test1.jpg Evaluation Metrics:** The post-processing script correctly predicted a human silhouette boundary, drawing a tight box with a classification confidence score of **78.1%**.



2. **test2.jpg Evaluation Metrics:** The model accurately handled a profile view of a seated user, drawing a bounding box over the torso with a localized confidence score of **59.4%**.



3. **test3.jpg Evaluation Metrics:** The model identified a central workspace setup, drawing a primary bounding box around a desktop monitor screen.



CHALLENGES FACED

1. **Network Architecture Output Proportions:** Standard classification backbones generate a single vector output. Designing a custom object detection head required structuring a unified output tensor that could handle a flat 10 x 4 continuous coordinate space alongside a 10 x 4 discrete classification label grid.
2. **Handling Padded Bounding Box Arrays:** Because different images contain different numbers of objects, batch training inputs must be padded to a uniform length. This introduces a challenge: empty padding slots can skew gradient updates. The issue was resolved by implementing a custom masked loss function that explicitly zeroes out gradients from unassigned labels.
3. **Spatial Average Alignment Shifts:** Early models trained with locked backbones tended to predict generalized, average bounding box locations rather than fitting tight boundaries. Unfreezing the top 40 convolutional layers allowed the deeper spatial feature maps to adapt, improving localization accuracy around distinct edges.
4. **Cloud WebSocket Port Firewall Blocks:** Google Colab's cloud security protocols block local background WebSocket connections. Running FiftyOne's interactive browser application inside a cell resulted in a Client is not connected error. This was resolved by writing custom Matplotlib inference loops to bypass the app and render test images directly in the notebook canvas.

CONCLUSION

Summary of Achievements

This project successfully designed and implemented a standalone, custom multi-task object detection pipeline built on top of a MobileNetV2 backbone. By using transfer learning and unfreezing the top 40 layers of the feature extractor , the network successfully optimized both multi-class categorical labels and bounding box regressions across a custom 1,500-image dataset split. Automated callbacks protected the network parameters from overfitting , ensuring the pipeline delivers clean predictions through a Non-Maximum Suppression filter.

Proposed Future Improvements

1. **Transition to an Advanced Localization Loss:** Replace the standard Mean Squared Error bounding box loss with an **Intersection over Union (IoU) Loss** or Generalized IoU (GIoU) function to improve alignment accuracy on overlapping edges.
2. **Integration of Anchor Boxes:** Migrate the current flat regression head to a structured anchor-box grid layout (such as an SSD or YOLO design) to better separate closely grouped objects.
3. **Real-Time Data Augmentation:** Introduce dynamic data augmentations during preprocessing—such as random crops, shifts, and color jitter—to increase the model's robustness against complex real-world shadows and lens glare.

REFERENCES

1. *Google Open Images V7 Repository & Annotation Protocols:*
<https://storage.googleapis.com/openimages/web/index.html>
2. *FiftyOne Vision Dataset Management API Documentation:*
<https://voxel51.com/docs/fiftyone/>
3. *MobileNetV2: Inverted Residuals and Linear Bottlenecks (Sandler et al.):*
[TensorFlow Applications Guide](#)
4. *Open Source Computer Vision Library (OpenCV) DNN Module Reference Manual:* <https://docs.opencv.org/>